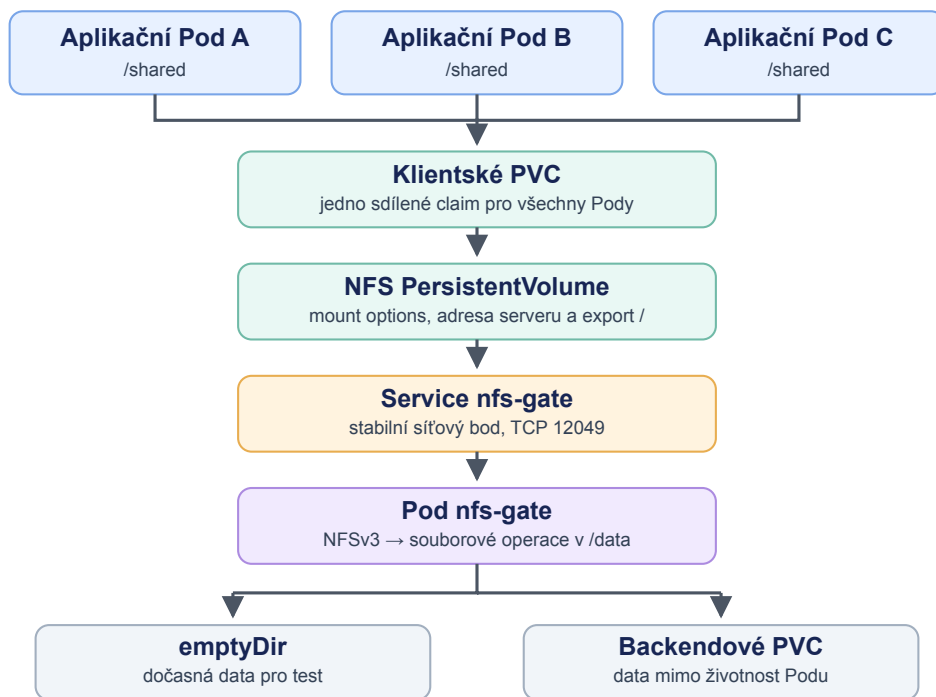


nfs-gate: společné soubory pro více Podů

Jednoduše řečeno

nfs-gate zpřístupní jeden adresář jako NFSv3 share. Více aplikací, klidně na různých uzlech Kubernetes, si tento share připojí jako běžný adresář. Když jedna aplikace vytvoří nebo změní soubor, ostatní pracují se stejným adresářem a změnu po aktualizaci své NFS cache uvidí.

Jak se soubor dostane ke všem Podům



Dole se vybírá jedna z možností. Klientské PVC není backendové PVC.

Obrázek 1: Tři aplikační Pody používají jedno klientské PVC. To vede přes NFS PV a Service do Podu nfs-gate, který exportuje emptyDir nebo backendové PVC.

Dvě různá úložiště: Klientské PVC je způsob, jakým si aplikace připojí NFS export. Naproti tomu backendový svazek je adresář, ze kterého nfs-gate data skutečně čte a do kterého zapisuje. Pro test může být backend emptyDir; pro data přesahující životnost serverového Podu může být backend samostatné PVC. nfs-gate žádné PVC automaticky nevytváří.

Co je vidět v UI: server má jednoduché read-only UI se stavem, posledními NFS operacemi a seznamem souborů. Obsah souborů nevydává. Historie zachycuje pouze operace, které prošly přes tento server; změny provedené přímo v backendu se objeví v seznamu souborů, ale ne v historii.

Důležitá hranice: emptyDir je vhodný pro ukázkou. Při odstranění nebo nahrazení serverového Podu se jeho data smažou. NFS klienti také mohou mít po restartu serveru neplatné file handle a potřebovat nové připojení. Stabilní adresa Service sama o sobě data ani handle nezachová. Kubernetes: emptyDir

Technické detaily

1. Server s backendem emptyDir

V testovací variantě má serverový Pod jeden adresář /data. Kubernetes ho naplní svazkem emptyDir a nfs-gate jej exportuje na TCP portu 12049. Následující části tvoří jeden Deployment a Service. Image musí být dostupný z registru daného clusteru.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nfs-gate
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels: {app: nfs-gate}
  template:
    metadata:
      labels: {app: nfs-gate}
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 65532
        runAsGroup: 65532
        fsGroup: 65532
      containers:
        - name: nfs-gate
          image: <registry>/nfs-gate:<tag>
          args: ["--root=/data", "--listen=:12049"]
          ports:
            - {name: nfs, containerPort: 12049}
          volumeMounts:
            - {name: data, mountPath: /data}
      volumes:
        - name: data
          emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: nfs-gate
spec:
  selector: {app: nfs-gate}
  ports:
    - {name: nfs, port: 12049, targetPort: nfs, protocol: TCP}
```

emptyDir přežije restart kontejneru ve stejném Podu, ale je odstraněn spolu s Podem. Je-li potřeba trvalejší backend, použijte krok 4. Kubernetes: emptyDir

2. Klientský PV a PVC

Klient potřebuje **statický NFS PersistentVolume**, protože mount používá nestandardní port. PV je objekt clusteru; PVC vzniká v namespace aplikací. Do pole server dosadte adresu, na kterou se dostanou *uzly* Kubernetes. V některých clusterech lze použít ClusterIP Service, pokud ji uzly směřují. DNS jméno *.svc.cluster.local je garantováno Podům, ne nutně samotným uzlům. Uzly také potřebují NFS mount helper.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs-gate-shared
spec:
  capacity: {storage: 1Gi}
  accessModes: [ReadWriteMany]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: ""
```

```

mountOptions:
  - nfsvers=3
  - proto=tcp
  - port=12049
  - mountvers=3
  - mountport=12049
  - mountproto=tcp
  - nolock
nfs:
  server: <adresa-dostupna-z-uzlu>
  path: /
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs-gate-shared
spec:
  accessModes: [ReadWriteMany]
  storageClassName: ""
  volumeName: nfs-gate-shared
  resources:
    requests: {storage: 1Gi}

```

storageClassName: "" brání tomu, aby se toto PVC svázalo s výchozí dynamickou StorageClass. 1Gi u NFS PV je deklarovaná kapacita pro Kubernetes, nikoli automatická kvóta. nolock nepoužívá rpc.statd; nfs-gate distribuované zámky neposkytuje. Mount options nelze zadat přímo do inline nfs: volume v Podu. Kubernetes: NFS volumes, PersistentVolumes

3. Tři klientské Pody

Všechny repliky používají **stejně klientské PVC**. Každá uvidí /shared a bude pracovat s jedním exportovaným adresářem. nfs-gate je stále jen jeden serverový Pod.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: shared-app
spec:
  replicas: 3
  selector:
    matchLabels: {app: shared-app}
  template:
    metadata:
      labels: {app: shared-app}
    spec:
      containers:
        - name: app
          image: <registry>/your-app:<tag>
          volumeMounts:
            - {name: shared, mountPath: /shared}
      volumes:
        - name: shared
          persistentVolumeClaim:
            claimName: nfs-gate-shared

```

Pro ověření stačí do /shared z jedné repliky zapsat soubor a z ostatních jej přečíst. Cache NFS klientů může krátce odložit viditelnost poslední změny.

4. Backendové PVC místo emptyDir

Pokud mají data přežít nahrazení serverového Podu, vytvořte pro server **druhé, odlišné PVC**. Jeho StorageClass a režim přístupu závisí na skutečném backendu. Pro jeden serverový Pod často postačí ReadWriteOnce; zvolený typ úložiště musí být připojitelný na uzlu, kam se Pod přesune.

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:

```

```
    name: nfs-gate-backend
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: <vase-storage-class>
  resources:
    requests: {storage: 10Gi}
```

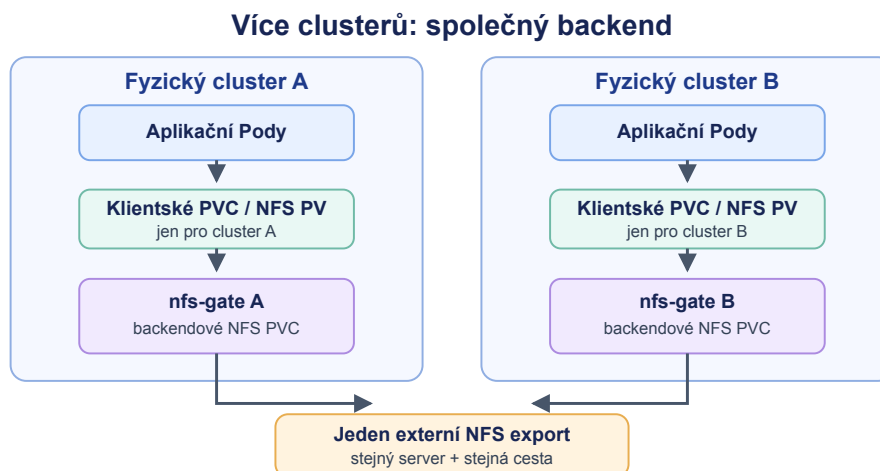
V serverovém Deploymentu nahradte pouze definici svazku data; volumeMounts na /data a --root=/data zůstanou:

```
volumes:
- name: data
  persistentVolumeClaim:
    claimName: nfs-gate-backend
```

Klientské PVC nfs-gate-shared se tím nemění. Backendové PVC chrání data podle vlastností svého úložiště, ale neřeší restart NFS serveru, dostupnost při výpadku ani platnost starých file handlů.

5. Více fyzických clusterů

Pokud běží samostatný nfs-gate v clusteru A i B a aplikace v obou clusterech musí vidět **jedny společné soubory**, musí mít oba servery pod /data **stejný externí NFS export a stejnou cestu**. emptyDir v každém clusteru je izolované a dvě běžná, nezávisle vytvořená PVC mohou ukazovat na různá data. Samotná shoda názvů PVC žádné sdílení nezajistí.



Obrázek 2: Dva clustery sdílejí jeden externí NFS export.

V **každém** clusteru vytvořte backendový statický NFS PV a PVC. Objekty PV/PVC jsou v clusterech nezávislé, ale hodnoty nfs.server a nfs.path musí ukazovat na stejný upstream export. Údaje v příkladu jsou zástupné:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs-gate-backend
spec:
  capacity: {storage: 10Gi}
  accessModes: [ReadWriteMany]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: ""
  nfs:
    server: <externi-nfs-server>
    path: /spolecny-export
---
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs-gate-backend
spec:
  accessModes: [ReadWriteMany]
  storageClassName: ""
  volumeName: nfs-gate-backend
  resources:
    requests: {storage: 10Gi}
```

V Deploymentu každého nfs-gate nahraďte emptyDir položkou persistentVolumeClaim: {claimName: nfs-gate-backend}. Kubelet připojí upstream NFS do Podu; proces nfs-gate ho čte jako běžný adresář. Klientský NFS PV/PVC z kroku 2 se v každém clusteru vytvoří zvlášť a míří na **místní** Service nfs-gate, nikoli přímo na upstream NFS server.

Zkontrolujte dosažitelnost upstream NFS z uzlů obou clusterů, oprávnění pro UID/GID serverového procesu a skutečnou exportovanou cestu. Dynamický NFS provisioner může každému PVC vytvořit jiný podadresář; pro sdílení je nutné, aby se oba backendové mounty opravdu shodovaly. Cache NFS může zpozdít změny mezi clustery a souběžné zápisy potřebují koordinaci v aplikaci. Společný upstream svazek zajistí společná data, ale sám o sobě neposkytuje HA pro nfs-gate ani stabilní file handle po restartu.

Provozní omezení

- NFSv3 handler používá AUTH_NULL. Kdo dosáhne na TCP 12049, může pracovat s exportem v rozsahu oprávnění procesu `nfs-gate`. Omezte přístup sítě a exportujte jen určený adresář.
- Server běží jako jedna instance. Další replika `nfs-gate` nad stejnými daty by nepřinesla stabilní file handle ani HA.
- Po restartu serveru může klient dostat `stale file handle`; může být nutné NFS znovu připojit. Tvrdé mounty mohou při výpadku čekat.
- Webové UI je read-only a bez autentizace. Výchozí nastavení ho váže na loopback; vzdálený přístup vyžaduje explicitní povolení a síťovou ochranu.

Kontrola po nasazení

1. Ověřte, že serverový Pod běží a klientské PVC je Bound. Pokud používáte backendové PVC, musí být Bound také ono.
2. Ověřte, že aplikační Pody skutečně nastartovaly s mountem `/shared`. Při chybě mountu zkontrolujte dostupnost serveru z uzlů, NFS mount helper a `mountOptions` klientského PV.
3. Z jedné aplikace vytvořte malý soubor v `/shared` a v další jej přečtěte. Krátké zpoždění může způsobit cache NFS klienta.
4. Pro více clusterů zopakujte čtení i v druhém clusteru. Pokud se data liší, ověřte, že backendová PVC obou serverů míří na **stejnou exportovanou cestu**.

Tento dokument používá zkrácené manifesty. Výchozí serverový manifest a úplný popis přepínačů jsou součástí projektu `nfs-gate`.